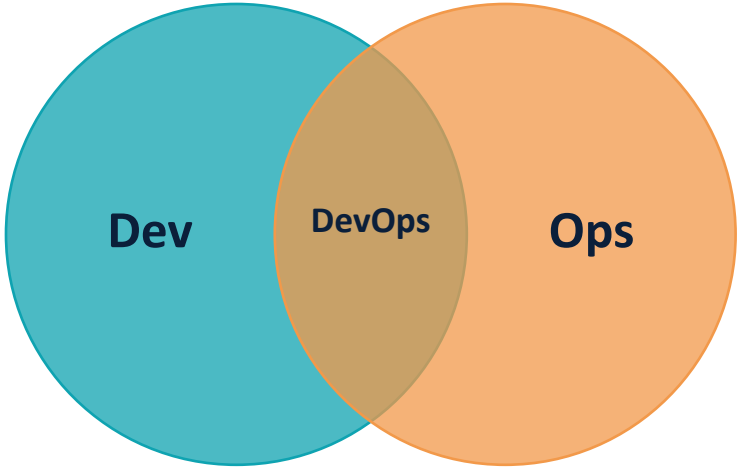
DevOps and CI/CD in Cloud Computing

Principles, pipeline architecture and practical implementation



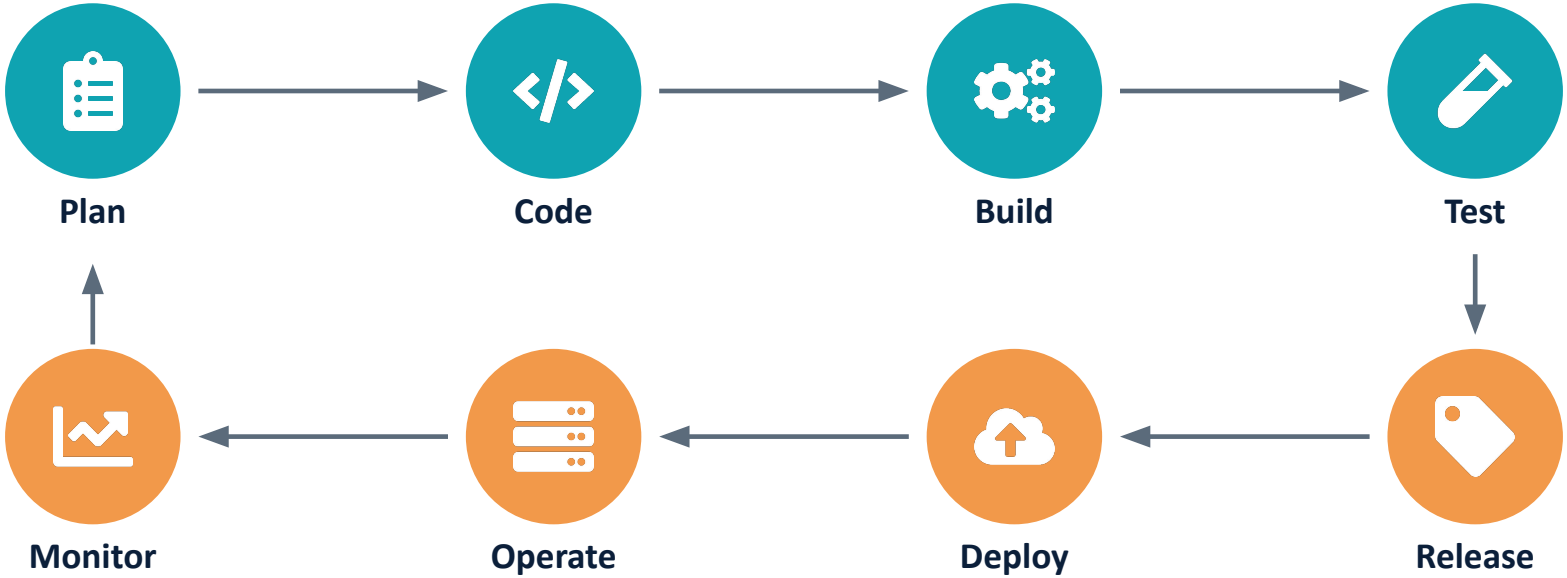
What is DevOps?

- A culture and set of practices that unites software development and IT operations
- Goal: deliver software faster, safer and more reliably
- Replaces silos and hand-offs with shared ownership
- Automates build, test, release and infrastructure
- Cloud supplies on-demand, API-driven resources that make this practical



Culture + Practices + Tools

The DevOps lifecycle



Continuous feedback: monitoring data flows back into planning

Core DevOps principles

Culture

Shared ownership across dev, ops and security

Automation

Script every repeatable build, test and deploy step

Lean flow

Small batches, fast delivery, less waste

Measurement

Track DORA metrics, logs and user impact

Sharing

Open knowledge and blameless post-mortems

Infrastructure as Code

Version-controlled, repeatable environments

Why DevOps thrives in the cloud



Elastic

Spin up build runners on demand, scale to zero



Pay-as-you-go

Pay only for pipeline minutes and resources used



Managed services

Registries and Kubernetes run by the provider



Global reach

Deploy to many regions in minutes

Cloud APIs + automation = environments created, tested and destroyed on every change

CI vs continuous delivery vs deployment

Continuous Integration	Continuous Delivery	Continuous Deployment
• Merge code to a shared repo often • Automated build on every commit • Unit tests and static analysis • Feedback within minutes	• Every change is release-ready • Automated integration and staging tests • Versioned artifacts stored • Manual approval for production	• No human gate • Passing builds go live automatically • Needs strong tests and monitoring • Automatic rollback on failure

Same abbreviation, increasing levels of automation

CI/CD pipeline architecture



Key architecture elements

- Trigger: webhook on push or pull request
- Artifact registry stores versioned images
- Secrets manager injects credentials at runtime
- Failures alert the team via chat or email

CI/CD services across clouds

Function	AWS	Azure	Google Cloud
Pipeline	CodePipeline	Azure Pipelines	Cloud Build triggers
Build and test	CodeBuild	Pipeline agents	Cloud Build
Artifact store	ECR / CodeArtifact	ACR / Azure Artifacts	Artifact Registry
Release	CodeDeploy	Release pipelines	Cloud Deploy
Runtime	ECS / EKS / Lambda	AKS / App Service	GKE / Cloud Run
IaC	CloudFormation / CDK	Bicep / ARM	Terraform / Infra Manager
Monitoring	CloudWatch	Azure Monitor	Cloud Monitoring

Jenkins, GitHub Actions, GitLab CI and Argo CD also run on all three clouds

Example: GitHub Actions pipeline

```yaml
name: ci-cd
on:
  push:
    branches: [main]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm test
      - run: docker build -t app:${{ github.sha }} .
  deploy:
    needs: build-test
    steps:
      - run: kubectl set image deploy/app
          app=app:${{ github.sha }}
```

What happens

1. Push to main starts the workflow
2. Tests run, then the image is built
3. Image is tagged with the commit SHA
4. Deploy runs only if tests pass
5. Kubernetes performs a rolling update

Deployment strategies

Rolling

Replace instances gradually, a few at a time.

Low cost, slower rollback

Blue-green

Run two identical environments and switch traffic at once.

Instant rollback, double cost

Canary

Send a small share of traffic to the new version first.

Safest, needs good monitoring

Feature flags and automated rollback complement all three

Practices that power the pipeline

Infrastructure as Code

Define networks and servers in Terraform or CloudFormation and review them like code.

Containers

Docker images run the same everywhere; Kubernetes orchestrates them.

DevSecOps

Scan code, dependencies and images inside the pipeline.

Observability

Metrics, logs and traces feed alerts and dashboards.

Best practices and challenges

Best practices

- Commit small and often
- Keep the pipeline fast
- Build once, deploy the same artifact
- Automate rollback
- Treat pipeline config as code

Common challenges

- Cultural resistance and silos
- Flaky or slow tests
- Secrets and compliance
- Tool sprawl and skill gaps
- Cloud cost control

Measure success with DORA metrics

Deploy frequency | **Lead time** | **Change failure rate** | **Time to restore**

Key takeaways



DevOps unites culture, automation and measurement



CI/CD turns every commit into a tested, deployable release



Cloud provides elastic, managed infrastructure for pipelines



Start small: automate build and test, then extend to deployment

Thank you | Questions?